Laboratório Prolog · Aula 13

Miguel Antonio Cruz Carreira · RA 24160032-2 · Turma B
Gerado em 20/10/2026, 20:09:20 · tempo registrado no cronômetro: 20:34

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

O paradigma(L, funcional) deve responder scheme, e paradigma(java, logico) deve dar false.

Saída do ola.pl

```
1 Saída:
Output:
--- constantes e variaveis
a  =  a    unifica
a  =  b    NAO unifica
A  =  a    unifica   A = a
--- estruturas
f(A,b)  =  f(a,B)    unifica   A = a   B = b
f(A,A)  =  f(a,b)    NAO unifica
f(A,A)  =  f(a,B)    unifica   A = a   B = a
pai(A,jake)  =  pai(jorge,B)    unifica   A = jorge   B = jake
--- listas: [Cabeca|Cauda]
[A|B]  =  [1,2,3]    unifica   A = 1   B = [2,3]
[A|B]  =  []    NAO unifica
[A,B|C]  =  [a,b,c,d]    unifica   A = a   B = b   C = [c,d]
--- aritmetica NAO e unificacao
2+3  =  5    NAO unifica

2 Saída:
--- constantes e variaveis
a  =  a    unifica
a  =  b    NAO unifica
A  =  a    unifica   A = a
--- estruturas
f(A,b)  =  f(a,B)    unifica   A = a   B = b
f(A,A)  =  f(a,b)    NAO unifica
f(A,A)  =  f(a,B)    unifica   A = a   B = a
pai(A,jake)  =  pai(jorge,B)    unifica   A = jorge   B = jake
--- listas: [Cabeca|Cauda]
[A|B]  =  [1,2,3]    unifica   A = 1   B = [2,3]
[A|B]  =  []    NAO unifica
[A,B|C]  =  [a,b,c,d]    unifica   A = a   B = b   C = [c,d]
--- aritmetica NAO e unificacao
2+3  =  5    NAO unifica
[A,B|C]  =  [1,2,3,4,5]    unifica   A = 1   B = 2   C = [3,4,5]
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

O paradigma(prolog, P) pergunta "qual é o paradigma do prolog?" e devolve o valor (logico). Já paradigma(prolog, logico) é uma pergunta de sim ou não: "o prolog é lógico?", e responde só verdadeiro ou falso.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Quer dizer que "o programa não sabe isso". O Prolog só conhece o que está na base, e não tem nenhum fato dizendo isso. Aqui até bate com a realidade, mas o false significa "não consegui provar", não "é mentira".

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

A mulher(Quem) deve ter 7 respostas. E (mae(M, F), mulher(F)) deve dar 4 pares: lucia e tereza, marta e joana, tereza e bia, joana e ana.

Saída do familia.pl

```
1 Saída:
Output:
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)
2 Saída:
Output:
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
```

```
    mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)
?- mae(joana,jake),mae(joana,ana).
   mae(joana,jake),mae(joana,ana)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

"Quais mães têm filhas mulheres?", ou seja, pares de mãe e filha. Deu 4 respostas: lucia/tereza, marta/joana, tereza/bia e joana/ana.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

É a hipótese do mundo fechado: o que não está na base e não dá pra provar é considerado falso. Na vida real isso pode dar problema porque "não sei" vira "não": por exemplo, um sistema pode dizer que um paciente não tem alergia só porque ninguém cadastrou.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

Saíram na ordem lucia, marta, tereza, joana, bia, ana, dora, que é exatamente a ordem em que os fatos estão escritos no código. O Prolog percorre a base de cima pra baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

O tio(X, jake) deve dar paulo (irmão da joana, mãe do jake). primo(leo, X) deve dar jake e ana, porque o pai do leo (paulo) é irmão da mãe deles (joana).

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Falham f(X, X) = f(a, b), [A|B] = [] e 2 + 3 = 5. Só f(X, X) = f(a, Y) unifica.

Saídas de regras.pl e unificacao.pl

```
1 Saída:
Output:
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
```

```
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
   primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
   ancestral(osvaldo,vern)
   ancestral(osvaldo,tereza)
   ancestral(osvaldo,joana)
   ancestral(osvaldo,paulo)
   ancestral(osvaldo,jake)
   ancestral(osvaldo,ana)
   ancestral(osvaldo,leo)
   ancestral(osvaldo,bia)
?- ancestral(A,jake).
   ancestral(joana,jake)
   ancestral(jorge,jake)
   ancestral(lucia,jake)
   ancestral(marta,jake)
   ancestral(osvaldo,jake)
   ancestral(vern,jake)
?- ancestral(jake,A).
   false.

2 Saída:
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
   primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
   ancestral(osvaldo,vern)
   ancestral(osvaldo,tereza)
   ancestral(osvaldo,joana)
   ancestral(osvaldo,paulo)
```

```
   ancestral(osvaldo,jake)
   ancestral(osvaldo,ana)
   ancestral(osvaldo,leo)
   ancestral(osvaldo,bia)
?- ancestral(A,jake).
   ancestral(joana,jake)
   ancestral(jorge,jake)
   ancestral(lucia,jake)
   ancestral(marta,jake)
   ancestral(osvaldo,jake)
   ancestral(vern,jake)
?- ancestral(jake,A).
   false.
?- irmao(A,B).
   irmao(vern,tereza)
   irmao(tereza,vern)
   irmao(joana,paulo)
   irmao(paulo,joana)
   irmao(jake,ana)
   irmao(ana,jake)
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Porque precisa de um ponto pra parar e de um passo que repete. O caso base é "progenitor direto é ancestral". O caso recursivo é "o progenitor de alguém que é meu ancestral também é meu ancestral".

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem o X \== Y, o irmao(jake, _) passa a responder jake e ana, ou seja, o jake vira irmão dele mesmo, já que ele tem a mesma mãe e o mesmo pai que ele próprio. O X \== Y serve pra tirar a própria pessoa.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

No f(X, X) = f(a, b), o X teria que ser a e b ao mesmo tempo, o que é impossível. No f(X, X) = f(a, Y), o X vira a e, como o Y precisa ser igual ao X, o Y também vira a. Fica X = a e Y = a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Em Java ou Scheme não, uma função tem entrada e saída fixas: você passa o neto e ela devolve os avós, e pra fazer o contrário teria que escrever outra função. No Prolog a regra é uma relação, não uma função, então dá pra deixar qualquer lado como variável e ele procura.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

A concatena(X, Y, [1, 2, 3]) deve dar 4 respostas, todas as formas de cortar a lista em duas: [] e [1,2,3], [1] e [2,3], [1,2] e [3], [1,2,3] e [].

Saída do listas.pl

```
1 Saída:Output:
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
   membro(b,[a,b,c])
?- membro(z,[a,b,c]).
   false.
?- membro(A,[a,b,c]).
   membro(a,[a,b,c])
```

```
   membro(b,[a,b,c])
   membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
   tamanho([a,b,c],3)
?- tamanho([],A).
   tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
   concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
   concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
   inverte([1,2,3],[3,2,1])
2 Saída:
Output:
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
   membro(b,[a,b,c])
?- membro(z,[a,b,c]).
   false.
?- membro(A,[a,b,c]).
   membro(a,[a,b,c])
   membro(b,[a,b,c])
   membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
   tamanho([a,b,c],3)
?- tamanho([],A).
   tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
   concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
   concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
   inverte([1,2,3],[3,2,1])
?- concatena(A,[b,c],[a,b,c]).
   concatena([a],[b,c],[a,b,c])
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

O caso base é tamanho([], 0), lista vazia tem tamanho 0. O passo recursivo é "o tamanho é o da cauda mais 1". Em tamanho([a, b, c], N) fica pendente o N is M + 1 de cada chamada: três somas esperando a volta, que só são feitas quando chega na lista vazia.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Porque existem 4 jeitos de dividir uma lista de 3 elementos em duas partes, e o Prolog acha todas por backtracking. O anexar do Scheme só junta duas listas que você já deu; ele não consegue fazer o contrário, tipo descobrir quais listas juntas dão [1, 2, 3].

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O backtracking. Ele acha a pela primeira cláusula, depois volta, tenta a segunda cláusula com a cauda e acha b, depois c.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

O X = 2 + 3 mostra a estrutura 2+3, sem calcular. X is 2 + 3 mostra 5. E fatorial(N, 120) acho que dá erro, porque o is não consegue calcular com N sem valor.

Saída do aritmetica.pl

```
1 Saída:
Output:
--- = unifica, is calcula
?- A=2+3.
   2+3=2+3
?- A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- A is 7 mod 3.
   1 is 7 mod 3
?- A is 7//2.
   3 is 7//2
?- A is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
   fatorial(5,120)
?- fatorial(0,A).
   fatorial(0,1)
?- fib(10,A).
   fib(10,55)
?- soma([1,2,3,4],A).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
   ERRO: instantiation_error
?- A is B+1.
   ERRO: instantiation_error
2 Saída:
--- = unifica, is calcula
```

```
?- A=2+3.
   2+3=2+3
?- A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- A is 7 mod 3.
   1 is 7 mod 3
?- A is 7//2.
   3 is 7//2
?- A is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
   fatorial(5,120)
?- fatorial(0,A).
   fatorial(0,1)
?- fib(10,A).
   fib(10,55)
?- soma([1,2,3,4],A).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
   ERRO: instantiation_error
?- A is B+1.
   ERRO: instantiation_error
?- A is 5*4+2.
   22 is 5*4+2
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

O = unifica, sem calcular nada: X = 2 + 3 deixa X valendo a estrutura 2+3, e 2 + 3 = 5 dá false. O is calcula o lado direito e liga ao esquerdo: X is 2 + 3 dá X = 5. O =:= compara os valores numéricos dos dois lados: 5 =:= 2 + 3 dá verdadeiro.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

Porque o is precisa do lado direito todo conhecido pra calcular. No fatorial(N, 120) o N é livre, então o N > 0 e o N1 is N - 1 dão instantiation_error. O concatena só usa unificação, que funciona com variáveis livres em qualquer lado. A aritmética não é relacional: só funciona numa direção.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

A ideia é a mesma: caso base com 0 dando 1 e o caso geral sendo N vezes o fatorial de N - 1. A diferença é que em Haskell o fatorial devolve o valor, e no Prolog o resultado entra como um argumento a mais (fatorial(N, F)). E no Prolog precisa do N > 0 e do is pra fazer as contas.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

O solteiro2(X) vai dar false. maximo2(5, 3, M) vai dar duas respostas (5 e 3). E maximo3(5, 3, 3) acho que dá true, mesmo estando errado.

Saída de controle.pl e mensagem de ancestral_ruim

```
1 Saída:
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(ana,caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio,ana)
   tenta caio e bia
   tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- solteiro2(A).
   false.
--- C) corte
?- maximo1(5,3,A).
   maximo1(5,3,5)
?- maximo2(5,3,A).
   maximo2(5,3,5)
   maximo2(5,3,3)
?- maximo3(5,3,A).
   maximo3(5,3,5)
?- maximo1(5,3,3).
   false.
?- maximo3(5,3,3).
   maximo3(5,3,3)

2 Saída:
ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)
Fatal Error: local stack overflow (size: 16384 Kb, reached: 16383 Kb, environment variable
used: LOCALSZ)
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores, eu chutaria umas 24 colorações. Com 2 cores, nenhuma, porque PR, SP e MS fazem fronteira entre si e precisam de 3 cores diferentes.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
   false.
```

```
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

Ele testou os pares na ordem: em "tenta ana e ana" quem falhou foi o X \== Y, aí ele voltou pro amigo(Y) e tentou o próximo, bia. Ana e bia falhou no gosto (café e chá), aí foi pra ana e caio, que deu certo: primeira resposta. Depois voltou de novo, trocou o X pra bia, testou todos, nada. Depois X = caio, e caio e ana deu a segunda resposta.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

No solteiro2, o \+ casado(X) roda com X sem valor. Aí o Prolog pergunta "existe alguém casado?", acha o vern, então a negação falha e tudo vira false. No solteiro1 o X já tem valor quando chega na negação, então funciona. E o osvaldo aparece como solteiro porque não tem o fato casado(osvaldo) na base. Ser pai não prova nada, é o mundo fechado de novo.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte só age se a primeira cláusula chegar até ele. Em maximo3(5, 3, 3), a cabeça maximo3(X, Y, X) já falha na unificação (precisaria 5 = 3), então nem chega no X >= Y, !. Aí ele vai pra segunda cláusula, que aceita qualquer coisa, e dá true. Correção: fazer a unificação depois do corte.
Corrigido:
maximo3(X, Y, Z) :- X >= Y, !, Z = X.
maximo3(_, Y, Y).

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

No ancestral_ruim, a primeira coisa que a regra faz é chamar ela mesma com os mesmos argumentos, sem avançar nada. Ela chama, chama, chama, até a pilha estourar (Fatal Error: local stack overflow). No ancestral_bom ela primeiro consome um progenitor e só depois chama de novo, então sempre anda um passo e uma hora termina. A lógica é a mesma, mas a ordem das metas muda tudo.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

Achou 24 colorações com 3 cores, nenhuma com 2 (deu false) e 1728 com 4. É declarativo porque o programa só descreve o que é uma coloração válida (cada região tem uma cor da paleta e vizinhas são diferentes). Não tem nenhum laço nem passo a passo, quem faz a busca é o backtracking do Prolog.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No funcional você escreve funções com entrada e saída fixas: o anexar do Scheme só junta duas listas. No lógico você descreve relações: o mesmo concatena junta, separa, testa e acha todas as formas de dividir uma lista, dependendo de onde estão as variáveis.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

Primeiro, a ordem das metas importa: o ancestral_ruim estourou a pilha sendo logicamente igual ao bom. Segundo, a negação como falha e o mundo fechado: solteiro2 deu false por causa da ordem, e o osvaldo virou "solteiro" só porque não tem fato dizendo que ele é casado.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog pra problemas de regras e busca: sistema especialista, regras de negócio complicadas, quebra-cabeças e restrições (tipo o mapa), consultas em árvore genealógica. Não escolheria pra sistemas com muita conta,

interface, desempenho ou coisa passo a passo, tipo um app web ou um jogo, onde Java é muito mais prático.